File Encryption Project

Project Report
Operating Systems Lecture Spring Semester 2026

University of Basel
Faculty of Science
Department of Mathematics and Computer Science

Nicolas Berger
Marvin Georges
Frédéric Weyssow
Simon Zeugin

June, 2026

Contents

1 Introduction

Ransomware is a type of malware that encrypts the victim's personal data until a ransom is paid. Ransomware attacks have caused billions of dollars in damages worldwide, affecting individuals, hospitals and other critical infrastructures. [1]
This project implements a ransomware simulation in a Linux environment, for educational purposes with the goal of understanding both the attack and the defensive mechanisms.
The simulation consists of three main components:

The Server/Client Architecture manages the encryption key transmission. The key is transmitted using RSA asymmetric encryption and then stored in memory in a masked form, ensuring the victim cannot retrieve it.

The Encryption Process, running entirely in user space, uses multithreading to encrypt files recursively from the /home directory as fast as possible, and monitors the file system for new files using inotify.

The Security Component, running in kernel space as a Loadable Kernel Module (LKM), intercepts file system operations via kprobes and detects processes with encryption behavior and alerts the user and gives them different ways to handle it.

This dual perspective, implementing both the attack and the defense, provides insight into how modern ransomware operates at the operating system level.

2 Background

2.1 Openssl

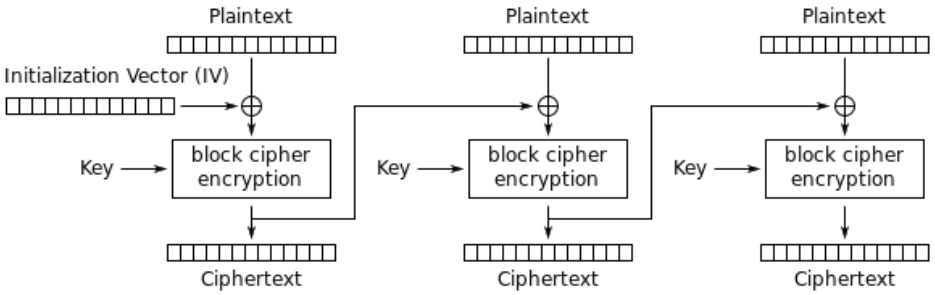
OpenSSL is an open-source cryptographic software library that offers implementations of the Transport Layer Security (TLS) protocol and a comprehensive set of cryptographic algorithms. It is widely deployed across servers, operating systems, and software applications to facilitate secure communications and digital certificate authentication.[2]

This project uses libcrypto, the cryptographic library provided by OpenSSL, for AES-CBC-128 file encryption and RSA-OAEP for key encryption.

2.1.1 File Encryption (AES-CBC-128)

This project uses the AES-CBC-128 algorithm provided by the OpenSSL API to encrypt files. AES (Advanced Encryption Standard) is a symmetric block cipher that operates on fixed-size 128-bit blocks. [3]
CBC stands for Cipher Block Chaining. As shown on the image below, each plaintext block is XORed[1] with the previous ciphertext block before encryption, creating a chain dependency. The first block uses a randomly generated Initialization Vector (IV) to ensure that identical plaintexts produce different ciphertexts. The IV is stored alongside the encrypted data to allow decryption.

[1] XORing the same value twice, returns the original data [4]

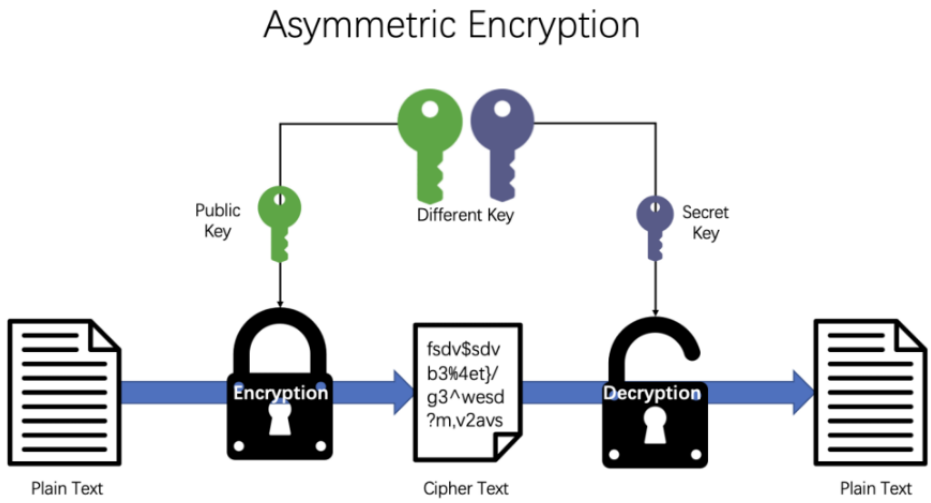


Cipher Block Chaining (CBC) mode encryption

[5]

2.1.2 Key Encryption (RSA-OAEP)

This project uses RSA (Rivest–Shamir–Adleman) to encrypt the encryption key. RSA is an asymmetric encryption algorithm that uses a mathematically linked key pair, a public key for encryption and a private key for decryption. This project uses RSA with OAEP (Optimal asymmetric encryption padding)(`RSA_PKCS1_OAEP_PADDING`), which is the recommended modern secure padding scheme by OpenSSL.[6]



[7]

2.2 raylib/raygui

Raylib is a simple open source library for creating graphical applications in C. Raygui is built on top of raylib. The GUI is newly rendered every frame.[8]
This project uses these libraries to display the GUI that appears after a successful encryption on the victim's screen and also for the security component to display the alerts.

2.3 Inotify

Inotify is a subsystem of the Linux kernel that allows user space programs to monitor changes in the filesystem. It can be used to detect modifications, creations, deletions and more. A program can add watchers to specific files or directories and receive notifications when something changes.[9]

2.4 Concepts for the Security Component

- **vfs_write** is a function of the Virtual File System in Linux. The Virtual File System is responsible for linking commands, like write() or read(), to the correct file system that handles the hardware. In this project we use vfs_write for linking specifically the write() command to the right file system. [10]

- **kprobes** is a dynamic tracing and debugging mechanism built into the kernel. It allows the user to intercept almost any kernel function in real-time without needing to recompile the operating system. When a kprobe is attached to a target function, the framework replaces the function's first instruction with a software breakpoint. When the CPU hits this breakpoint, normal execution is paused, and the kernel diverts the flow to a custom, user-defined function before safely resuming to the original task. [11]

- **Netlink Sockets** are a interprocess communication mechanism used in Linux. While they use the standard network socket programming interface, they are designed for high speed, bidirectional data transfer between the kernel and user space applications, routing messages internally using process IDs rather than network addresses. [12] [13]

3 Methodology

3.1 Phishing

To gain initial access to the system, we used a phishing strategy to trick the user into executing our malware. We disguised the malware as a simple, harmless calculator. For our distribution method, we presented the file as a beginner's programming project and asked the victim for feedback and coding advice to lower their suspicion.
Once the program is executed, it presents the user with a calculator on a separate thread. At the same moment the actual malware initiates in the background.

3.2 Server/Client architecture

We implemented a classic server/client architecture using TCP sockets. The server listens on port 8080 and maintains a queue of connected clients, using the socket file descriptor as a unique identifier for each connection.[14]

3.2.1 Server

The server runs two main concurrent threads, plus one additional thread per connected client:

- **connection_listener** waits for incoming client connections using accept(). When a new client connects, it is added to socket_queue and a dedicated handle_receive_message thread is created for that client. This thread continuously reads incoming messages using read() and removes the client from the queue upon disconnection.

- **commands_listener** reads commands from standard input using fgets(). Each command is parsed by splitting on the semicolon delimiter using strtok(), producing a command and an optional parameter.

The server maintains a linked list queue socket_queue of connected clients, protected by a mutex to prevent race conditions.

3.2.2 Client

The client connects to the server via TCP using the server's IP address on port 8080. Upon connection, the client sends an identification message containing system information gathered using uname() and /sys/devices/virtual/dmi/id/product_name:

```
PC connected, number of file: 33, model: ThinkPad-X1,
username: usrname_example, OS kernel: Linux
```

The server then immediately responds with the RSA-encrypted AES key, which the client decrypts and stores securely in memory as described in section 3.3.

The client runs two concurrent threads:

- **commands_listener** continuously reads commands from the server using read(). Commands are parsed by splitting on the semicolon delimiter using strtok(), then dispatched using if/else statements:
  encrypt clears the file queue, traverses the /home directory, populates the queue and launches the encryption threads.
  decrypt same as encrypt but in reverse, launching the decryption threads.
  kill retrieves its own executable path via /proc/self/exe, deletes it using unlink(), then exits.
- **start_watcher** monitors the file system for new files using inotify, described in section 3.5.

3.2.3 Network Protocol

The server communicates with clients through a simple protocol. Commands are separated by semicolons, following the format `command;parameter`.

Command	Parameter	Description
encrypt	socketfd / all	Encrypt all files
decrypt	socketfd / all	Decrypt all files
kill	socketfd / all	Terminate the client process
list		List all current client connected

3.2.4 Key Transmission

As shown in the Results section 4, figure 1 and 2. The key is encrypted during the TCP communication to avoid the raw key to appear in plaintext in the traffic and be detected using tools like wireshark.

The RSA key pair was generated using the OpenSSL command-line tool:

```
openssl genrsa -out private.pem 2048
openssl rsa -in private.pem -pubout -out public.pem
```

The server holds two hardcoded values: the 16-byte AES key and the RSA public key in PEM format. When a client connects, the server encrypts the AES key using the public key via OpenSSL `EVP_PKEY_CTX` API with `RSA_PKCS1_OAEP_PADDING` and immediately sends the 256-byte encrypted result over the TCP connection:

```
1 EVP_PKEY_CTX_set_rsa_padding(ctx, RSA_PKCS1_OAEP_PADDING);
2 EVP_PKEY_encrypt(ctx, out, out_len, aes_key, key_len);
3 send(client_fd, encrypted, enc_len, 0);
```

On the client side, the 256 encrypted bytes are received and decrypted using the hardcoded RSA private key, also via `EVP_PKEY_CTX` with the same padding scheme:

```
recv(client_fd, encrypted, sizeof(encrypted), 0);
EVP_PKEY_decrypt(ctx, raw_key, &raw_len, encrypted, enc_len);
```

3.3 Key Management

To keep the key hidden for the victims. The raw key never persists in memory. After the key decryption, it is immediately XORed with a randomly generated mask:

```
RAND_bytes(key.mask, 16);
for (int i = 0; i < 16; i++) {
    key.masked_key[i] = raw_key[i] ^ key.mask[i];
}
mlock(&key, sizeof(key));
OPENSSL_cleanse(raw_key, 16);
```

Both the masked key and the mask are stored separately in memory, locked from being swapped to disk using mlock() and the raw key buffer is wiped immediately using OPENSSL_cleanse().

Each time the key is needed for an encryption or decryption operation, it is reconstructed temporarily in a local buffer, used and immediately wiped:

```
for (int i = 0; i < 16; i++) {
    out[i] = key.masked_key[i] ^ key.mask[i];
}
// use tmp_key for AES operation
OPENSSL_cleanse(tmp_key, 16)
```

This ensures the plaintext key exists in memory for only a short time during each file operation.[4]

3.4 En-/Decryption

3.4.1 Encryption

The first step of encryption is to generate the initialization vector. This vector is a randomly generated 16-byte array, which is needed to ensure that encrypting the same file twice produces a different ciphertext, preventing pattern recognition.

```
unsigned char iv[16];
RAND_bytes(iv, sizeof(iv));
```

Then we set up an OpenSSL cipher context and use it for EVP_EncryptInit_ex, passing the context, the algorithm (AES-128-CBC), the default engine, the key and the initialization vector.

```
EVP_CIPHER_CTX* ctx = EVP_CIPHER_CTX_new();
EVP_EncryptInit_ex(ctx, EVP_aes_128_cbc(), NULL, key, iv);
```

The file is read chunks wise with fread(). The buffer dynamically grows with realloc() as needed, while all the encrypted chunks get appended one by one.

```
while ((bytes = fread(temp, 1, sizeof(temp), in)) > 0) {
    if (size + bytes + 16 > capacity) {
        capacity = (capacity + bytes) * 2;
        char *tmp = realloc(buffer, capacity);
        buffer = tmp;
    }
```

```
    EVP_EncryptUpdate(ctx, buffer + size, &outlen, temp, bytes);
    size += outlen;
}
```

Once all chunks are encrypted and stored inside the buffer, the original file is reopened to be overwritten. The initialization vector is written first in the file as a header, since it will be needed to decrypt the file. The EVP_EncryptFinal_ex handles the final block, applying padding to fill it to a full 16 bytes. After that buffer is then written into the file.

```
fwrite(iv, 1, 16, out);
EVP_EncryptFinal_ex(ctx, buffer + size, &outlen);
fwrite(buffer, 1, size + outlen, out);
```

We did not have to create a new file and delete the old one, since the file is overwritten rather than replaced. Finally the file is renamed to <filename.type>.locked using rename().[15][16]

```
char *new_name = malloc(strlen(file) + 8);
snprintf(new_name, strlen(file) + 8, "%s.locked", file);
rename(file, new_name);
```

3.4.2 Decryption

Similar to encryption, the initialization vector is read from the first 16 bytes of the file, after that the remaining content is read chunk wise, then EVP_DecryptUpdate is called. Once the buffer contains the fully decrypted content, the file is overwritten with the original content. In the end the .locked extension is stripped from the filename and the file is usable again.[15][16]

3.4.3 Initial traversal

During an encryption, traverse() is the first function called. While traversing the directory structure, all file paths are enqueued for later encryption.
The first step of the traverse function is to open the given directory (/home). Then each entry is checked against our is_skipped list (3.5), to skip folders like venv or .git which should not be enqueued. The full path is then built with the name we get from readdir() and the path of the current folder. Afterward, lstat() is called to determine the type (file or folder) of the entry. We either enqueue the file or call traverse() again, making the function recursive.

```
if (S_ISDIR(statbuf.st_mode)) {
    traverse(fullPath);  // recursive call
} else if (S_ISREG(statbuf.st_mode)) {
    enqueue(&queue, fullPath);
}
```

[17]

3.4.4 File Queue

The files that are needed to be encrypted or decrypted are managed through a linked list queue. Each node stores a file path string. The queue is protected by a mutex to prevent race conditions between the threads consuming files and the watcher(3.5) producing new ones.
After each traversal, a sentinel value (`END_ENCRYPT` or `END_DECRYPT`) is appended to the tail of the queue to signal the end.

3.4.5 Multithreading

To improve speed and performance, we encrypt files parallel using a defined number of threads. Once an en-/decrypt command is received, traverse() is called and the queue is filled.
The threads are all starting the same runner() function. This function dispatches them either to start_encrypt() or start_decrypt() the files in the queue.

```
void *runner(void *arg) {
    if (*(bool *)arg == true) {
        start_encrypt();
    } else {
        start_decrypt();
    }
    return NULL;
}
```

Each thread then uses the peek_and_dequeue() function, which automatically checks for sentinel value and dequeues the next path in a single locked operation, thus preventing race conditions between threads.

```
while (true) {
        char *data = peek_and_dequeue(&queue, "END_ENCRYPT");
        if (data == NULL){
            break;
        }
        unsigned char key[16];
        use_key(key);
        char *filename = encrypt_file(data, key);
        wipe_key(key);
        free(data);
}
```

3.5 Watcher

The watcher is tasked to encrypt newly added files using inotify. It is active after the server sends the encrypt command, so the victim cannot work in the corrupted directory.
To use inotify, we first initialize it with inotify_init(), which creates an inotify instance in the kernel and returns a file descriptor:

```
int fd = inotify_init();
```

We then register watchers on directories using inotify_add_watch() with events these flags:

```
int wd = inotify_add_watch(fd, path, IN_CREATE | IN_MOVED_TO |
   IN_CLOSE_WRITE);
```

The file descriptor is then read() in an infinite loop. Since, reading is blocked, the loop waits until an event actually occurs, rather than burning CPU cycles with no progress:

```
while (true) {
    int len = read(fd, buf, BUF_LEN);
    while (i < len) {
        struct inotify_event *event = (struct inotify_event *)&buf[i];
        // handle event...
        i += sizeof(struct inotify_event) + event->len;
    }
}
```

Recursive Directory Watching: Since we wanted to watch all subdirectories of /home, we implemented watcher_traverse(), which recursively registers watchers on every subdirectory. We store each watch descriptor alongside its corresponding path, since the descriptor alone does not tell us which path it watches. We use lstat() instead of stat() to avoid following symbolic links, which caused problems in earlier versions.

Event Handling: When a new file is detected, we encrypt it and add it to the queue. When a new directory is detected, we call watcher_traverse() on it recursively. In the past we edited the queue individually per event, which turned out to be inconsistent. We chose to always rebuild the queue from scratch before any encryption, which is less efficient but much more reliable.

Skip List: The is_skipped() function prevents watcher_traverse() from entering folders like .git, node_modules and venv. Without this, inotify_add_watch() would eventually return -1 which leads to an out-of-bounds write and segmentation fault.

watcher_on Flag and Buffer Drain: The watcher is controlled with a watcher_on boolean flag. We turn it off before decryption begins to prevent the watcher from interfering with files being decrypted. We also had to implement a buffer drain using poll(), since Linux still buffers inotify events even when the watcher is off. Without draining, those buffered events would trigger unwanted encryptions when the watcher was re-enabled. [9][18][19]

3.6 Security Component

3.6.1 Idea

The goal of the security component is to monitor the file write activity. It supervises the file system starting from the /home directory. The component is designed to detect suspicious behavior and to notify the user with an alert when such activity is found. After an alert is shown, the user can decide if the suspicious process should be killed, continued or trusted. Suspicious activity is defined if one process changes multiple files in different directories in a short defined time.

3.6.2 Kernel Space Monitoring

We implemented the detection logic as a Linux kernel module (LKM). When the module is loaded, the kernel calls our function start_function() through module_init().[20] In this function, we initialize the netlink communication and register our kprobe. After the module is loaded, the actual monitoring starts through the registered kprobe. When the module is removed, the kernel calls our function exit_function() through module_exit(). This function unregisters the kprobe and releases the netlink socket. By running the detection logic inside the kernel, we can observe file write operations.

Kprobe and vfs_write:

```
1 static struct kprobe kp = {
2     .symbol_name = "vfs_write",
3     .pre_handler = handler_pre, };
```

We attach the kprobe to the kernel function vfs_write. This function is part of the Linux Virtual File System and is called when a process writes data to a file. We use this function as our monitor because it allows us to observe file write operations directly inside the kernel.

In our implementation, the kprobe is registered when the kernel module is loaded. The kprobe

uses a pre_handler function that is executed before vfs_write continues. This means that every time a process tries to write to a file, our code can first inspect the write operation.

Inside the pre_handler function, we identify the process that caused the write operation. We get the process ID with current → pid and the process name with current → comm. We also access the file that is being written to and take the full file path. This allows the security component to know, which process is writing to which file and in what directory and keeps track to find suspicious behaviour.

Suspicious activity detection: After the pre_handler has identified the process and the file path, the detection logic checks whether this write operation should be monitored. First, the module checks whether the PID or the program path is already trusted. If the PID or path is trusted, the write operation is ignored by the detection logic.

For every monitored process, the module stores an entry in a PID list. This entry contains the process ID, a timestamp, the number of unique files, and the number of unique directories that the process has written to. The monitoring is done per PID, so each process is counted separately.

When a process writes to a file, the module checks if the written file path is already stored for this PID. If the file path is new, the unique file counter is increased. The module also extracts the directory from the file path. If this directory is new for this PID, the unique directory counter is increased. The counters are only valid for a short time window defined by TIME_WINDOW_MS. If the time window has passed, the counters and stored paths for the process are reset. This prevents old write operations from being counted forever.

A process is marked as suspicious when it reaches both limits (unique file limit and unique directory limit) inside the time window. This behavior is suspicious because ransomware often modifies many different files in different directories in a short amount of time.

When this condition is reached, the module marks the process as suspicious and stores the suspicious PID and process name. If no alert is currently pending, the process is stored as the current active alert. If another alert is already pending, the process is stored in the alert queue. The process entry is then deactivated, so the same entry can be reused later. At this point, the detection part is finished. The suspicious process is then passed to the process control and alert handling logic, which decides what should happen next.

Process Control: When we detect a process as suspicious, the kernel module first stops the process by sending a SIGSTOP signal. This does not kill the process immediately, it only pauses the process so that it cannot continue overwriting more files.
After the process is stopped, we create an alert for the user space program. The user can then choose how the suspicious process should be handled. If the user chooses to kill the process, the kernel sends SIGKILL to terminate it. If the user chooses to continue the process, the kernel sends SIGCONT, which allows the stopped process to continue running. The user can also choose to trust the process. If the PID is trusted, this specific running process is ignored by the detection logic in future checks. If the program path is trusted, future processes started from the same path are also ignored. This is useful for harmless programs that also meet the conditions we set for suspicious activity and therefore should not trigger repeated alerts.

Communication with User Space: We use netlink to communicate between kernel mod-

ule and user space program. It serves as our IPC tool. This is needed because the kernel module cannot directly interact with the graphical user interface, instead the kernel module sends alert messages to the user space, to handle interactions with the user.

In the kernel module, we create a netlink socket when the module is loaded. This is done in start_function() with netlink_kernel_create(). We also define a callback function called nl_recv_msg(). This function is called automatically when the kernel receives a netlink message from user space.

```
struct netlink_kernel_cfg cfg = {
        .input = nl_recv_msg,
};

nl_sk = netlink_kernel_create(&init_net, NETLINK_SECURITY, &cfg);
```

The user space controller (user_controller.c) also creates a netlink socket. This is done with the normal socket API by calling
socket(AF_NETLINK, SOCK_RAW, NETLINK _SECURITY).[12] The controller then binds the socket to its own process ID with bind(). This value is important because the kernel stores it as the user space port ID and uses it as the destination when sending messages back to the user space.

```
nl_sock = socket(AF_NETLINK, SOCK_RAW, NETLINK_SECURITY);

local_addr.nl_family = AF_NETLINK;
local_addr.nl_pid = getpid();

bind(nl_sock, (struct sockaddr *)&local_addr, sizeof(local_addr));
```

After the user space socket is created, the controller sends a REGISTER message to the kernel. This first message tells the kernel which user space process is listening. In the kernel module, the nl_recv_msg function reads the sender information from the received message and stores the user space port ID. The kernel needs this user_portid so it knows where to send future alert messages. Without this registration step, the kernel would not know which user space process should receive the alerts. When we want to send an alert from the kernel to user space, we first build a text message. If there is an active alert, the message contains the PID, process name, and program path. If there is no active alert, the kernel sends NO_ALERT. The kernel then creates a netlink message buffer with nlmsg_new(), adds a netlink header with nlmsg_put(), copies the message into the netlink payload, and sends it to the user space controller with nlmsg_unicast().

On the user space side, the controller receives netlink messages with recvmsg(). The received message contains a netlink header and a payload. The controller reads the payload with NLMSG_DATA() and parses the alert message. This parsed alert is then used by the interface to show the suspicious process to the user. The communication is bidirectional. When the user chooses an action in the GUI, the user space controller sends a command string back to the kernel with sendmsg(). The kernel then receives these commands in nl_recv_msg and then performs the correct action.

Synchronization: For protecting critical sections we have to use spinlocks. Because we use multiple threads for our encryption, multiple CPU cores can trigger the kprobe at the same time. If the kprobe is triggered by multiple threads concurrently, it could attempt to modify the tracking lists and the alert queue simultaneously. This could lead to a race condition and crash the kernel. With a spinlock, the locked thread never goes to sleep but rather is trapped

in a busy wait loop until the lock becomes available. We used this because our detection logic has to be fast, and sleeping inside a kprobe handler could lead to severe system instability since it executes in an atomic interrupt context where sleeping is strictly forbidden.
We use spin_lock_irqsave and spin_unlock_irqrestore. With this variant, we not only acquire the lock, but we also disable local hardware interrupts on the current CPU. We apply this to protect three critical sections in our code:

- The pid_lock protects the pid_list array and the pid_activity data structure. These structures are responsible for tracking how many unique files and directories have been modified by a program within a defined time window.
  With the lock, we ensure that the tracker updates the counters one by one, preventing threads from overwriting the counter simultaneously and corrupting the tracking data.

- The alert_lock protects the alert_queue array and the active alert state variables. The alert_queue holds the list of suspicious programs waiting to be sent to the user interface.
  In this case, we rely on a producer-consumer model. The kprobe handler acts as the producer by pushing suspicious programs into the queue, and the netlink receive callback acts as the consumer by processing commands from user-space.
  The alert_lock prevents the queue from becoming corrupted when these two actions happen at once.

- The allowed_pid_lock and allowed_PID_path_lock protect the dynamic arrays that store trusted process IDs and executable paths.
  We rely on these locks to ensure that if our allowlist is actively being updated by the user, the kprobe does not attempt to read from it at the exact same time.

3.6.3 User Controller

We use the user controller to connect the security GUI, implemented in `security_gui.c`, with the security component. The user controller provides the functions that the GUI can call to receive alert information from the kernel module and to send the user's decision back to it. The user controller stores received alert information in a SecurityAlert structure. This structure stores the alert state, the suspicious PID, the process name, and the executable path. The GUI uses this information to show the alert to the user and gives them three different options:

- **Kill** option terminates the process.

- **Trust** option adds the process to the allowed path list, so future processes started from the same path are ignored by the detection logic.

- **Continue** option allows the stopped process to continue running but it can still trigger an alert if it becomes suspicious again.

When an option is chosen, the corresponding command is sent to the kernel module, which then executes the selected action.

4 Results

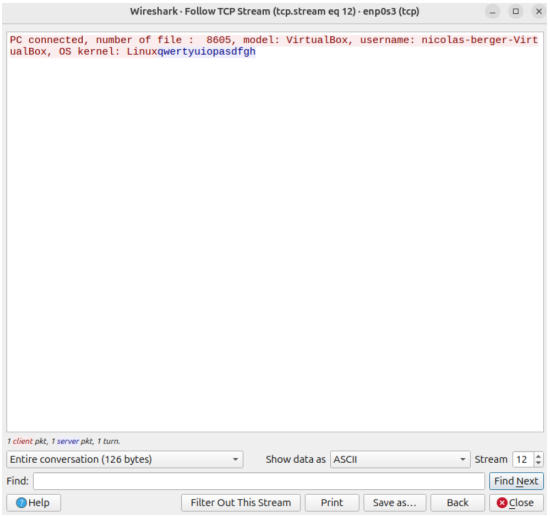


Figure 1: Wireshark: Plain text key

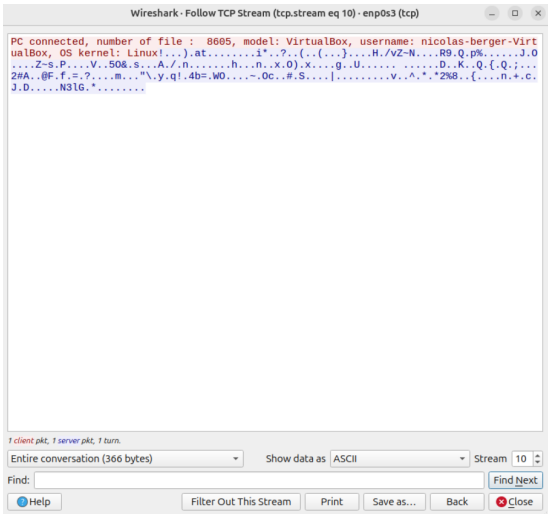


Figure 2: Wireshark: Key encrypted

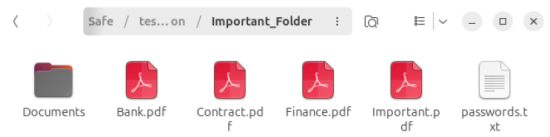


Figure 3: Before encryption

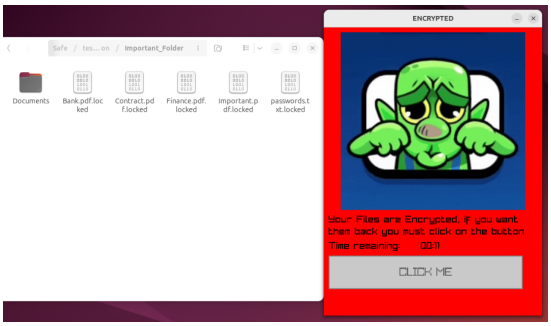


Figure 4: After encryption

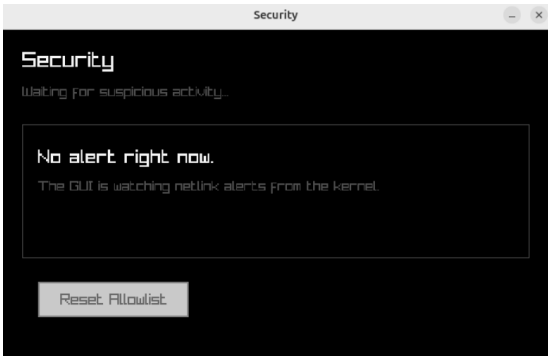


Figure 5: Detection: Before encryption

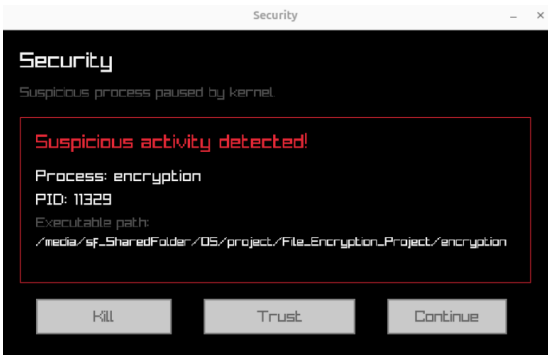


Figure 6: Detection: During encryption

We tested our speed in respect to amount of threads for encryption and decryption of over 900 files.

Threads	Encryption time	Decryption time
2	13 seconds	11 seconds
4	10 seconds	8 seconds
8	7 seconds	5 seconds

5 Discussion

5.1 Achievements

Encryption: We successfully implemented a ransomware simulation that recursively traverses the file system (starting from /home) and encrypts the files in it. Additionally, we implemented a server/client architecture using TCP for the key exchange. The key management permits the key to never persist in the memory.
The watcher can continuously encrypt new files and monitor newly created directories using inotify. This means that files created after the initial encryption are also detected and encrypted.

Multithreading: Using multithreading improved the speed of our encryption and decryption processes. However, generating too many threads is inefficient and causes high CPU overhead as the processor struggles to manage them all. Instead, we identified a moderate threshold that successfully balances maximum execution speed with stable CPU usage.
The results table demonstrates that encryption and decryption times vary slightly across different runs. This fluctuation occurs because the overall system performance and CPU load naturally shift during each individual test execution.

Security Component: Our security component can successfully monitor most processes and track which files and directories they write to. It stores this information for each process and uses it to decide whether the behavior matches our definition of a suspicious activity. It can also communicate successfully through netlink. This allows the kernel module to send alert information to the user controller and receive the selected user action in return. We also implemented a working GUI. The GUI uses the functions provided by the user controller and displays them correctly.

5.2 Limitations

Server/Client: Since the server has no domain name or public hosting, it is only reachable on localhost or within a shared local network. Also the server IP address is hardcoded directly in the client binary. This is a necessary design choice since the client must know where to connect at startup, without any user interaction. However this means the binary must be recompiled each time the server IP address changes.

Encryption: If the encryption process does not have sufficient permissions to read or write a file, that file is skipped. This means files owned by other users or protected by restrictive permissions would not be affected by the encryption.

Security component: The security component cannot detect an encryption operation instantaneously. Before the system can accurately classify a behavior as suspicious, the target process must reach the predefined limit of modified unique paths and directories. The inherent problem with this time window approach is that by the time the threshold is met, some files will have already been successfully encrypted.
The security component can also produce false alarms, because the detection is based on write patterns. A harmless program can also write many files in different directories within a short time. In addition, our tool does not provide a mechanism to recover them. Despite this initial data loss, our security component effectively halts the malware, preventing the destruction of whole directories and securing the rest of the files.

Watcher: We encountered problems with special directories such as .git or venv. These contain a very large number of subdirectories and files, which flooded the inotify buffer and caused unwanted encryptions. To address this, is_skipped(const char *name) is called on every event name and directory entry, silently ignoring anything that matches the list. There will certainly be edge cases with other directory structures that are not covered, but since it is impossible to test every file type and folder layout, only the most common ones are handled. The list can easily be extended if new problems arise.

6 Conclusion

6.1 Summary

The original project proposal was to simulate a ransomware attack consisting of multi-threaded file encryption, a TCP server to manage multiple victims, and a monitoring component to detect the encryption process. Looking back at this proposal, we achieved the core goals in a working state.
The core encryption and decryption using AES-128-CBC via Open SSL was fully implemented with multithreading as planned. The server successfully manages multiple clients simultaneously.
The monitoring component originally proposed two options, a defender using fanotify or an accomplice using inotify. We decided to implement both. Inotify is used on the client side by the watcher to monitor new files during encryption. We implemented first a working defender using fanotify but decided later to make a defender which acts on the kernel layer without the help of fanotify. The reason for this change was that we found it more interesting to program at the kernel level and the defender would also be more powerful and efficient.

6.2 Future Work

An improvement for the server/client architecture would be removing the hardcoded server IP address in favor of dynamic detection. We had the idea of using a middleman service, specifically a bot (Telegram, Discord), to connect the client with the attacker. We chose not to implement this architecture for safety reasons because then the project could theoretically be used as a functional malware in the real world.

We considered creating snapshots of the whole file system when the security component starts. With that idea, the corrupted file system could potentially be restored from these "snapshots". In general, the detection of suspicious activities can always be improved, so that our security component could better detect encryptions or other dangerous activities.

6.3 Lessons Learned

Through this project we learned many new technical skills including multithreading, TCP socket programming, cryptography with OpenSSL, file system monitoring with inotify, and Linux kernel module development with kprobes. Writing the kernel code was the most challenging part, because one small mistake could endanger the whole operating system.
Overall, our time management was highly effective. By beginning development early, we successfully avoided major time pressure as the deadline approached. One minor scheduling challenge arose from our decision to redesign the security component. Ultimately, however, the final outcome met our expectations, and we are satisfied with the results.
Our teamwork remained strong throughout the entire project. We consistently held weekly

meetings to discuss our progress and we continuously collaborated to troubleshoot and resolve issues.

References

[1] Matthew Kosinski, Stephanie Susnjara, Michael Goodwin, "What is ransomware?."

[2] Wikipedia contributors, "openssl," 2026.

[3] Wikipedia contributors, "Block cipher mode of operation." https://en.wikipedia.org/wiki/Block_cipher_mode_of_operation, 2026. Wikipedia, The Free Encyclopedia.

[4] Murray Distributed Technologies, "Masking and the use of XOR for encryption and decryption." https://mdtechnologies.medium.com/masking-and-the-use-of-xor-for-encryption-and-decryption-4a0a4a37c51a, February 2022. Published: February 23, 2022.

[5] Derek Will, "AES CBC Mode Chosen Plaintext Attack." https://derekwill.com/2021/01/01/aes-cbc-mode-chosen-plaintext-attack/.

[6] Wikipedia contributors, "Optimal asymmetric encryption padding." https://en.wikipedia.org/wiki/Optimal_asymmetric_encryption_padding, 2026. Wikipedia, The Free Encyclopedia.

[7] Y. Zhong, "An Overview of RSA and OAEP Padding," in *Highlights in Science, Engineering and Technology, ICCIA 2022*, vol. 1, pp. 82–86, 2022.

[8] R. Santamaria and contributors, "raygui: A simple and easy-to-use immediate-mode gui library," 2023.

[9] man7.org, *inotify(7) Linux manual page*. Linux Programmer's Manual.

[10] R. Gooch, *Overview of the Linux Virtual File System*. The Linux Kernel Documentation.

[11] M. H. Jim Keniston, Prasanna S Panchamukhi, *Kernel Probes (Kprobes)*. The Linux Kernel Documentation.

[12] The Linux Kernel Documentation, *Introduction to Netlink*.

[13] K. Kaichuan, *Kernel Korner Why and How to Use Netlink Socket*. Linux Journal. Accessed: 2026-08-06.

[14] GeeksforGeeks, "TCP server-client implementation in C." https://www.geeksforgeeks.org/c/tcp-server-client-implementation-in-c/, 2025. Last updated: July 11, 2025.

[15] The OpenSSL Project Authors, "EVP_EncryptInit, EVP_EncryptUpdate, EVP_EncryptFinal and related functions." OpenSSL Documentation.

[16] The OpenSSL Project Authors, "EVP_CIPHER-AES – the AES EVP_CIPHER implementations." OpenSSL Documentation.

[17] J. Robbins, "count_lines.c recursive directory traversal in C." https://github.com/jackr276/Traversing-Directories-in-C/blob/main/src/count_lines.c, 2024. Part of the Traversing-Directories-in-C repository.

[18] Wikipedia contributors, "Inotify," 2026.

[19] Daniel Hirsch, “File watcher in c.” YouTube, 2026.

[20] The Linux Kernel Documentation, *Driver Basics*.

[21] Swiss Federal Council, “Art. 143 stgb unbefugte datenbeschaffung.” https://www.fedlex.admin.ch/eli/cc/54/757_781_799/de#art_143, 2024.

[22] Swiss Federal Council, “Art. 143bis stgb unbefugtes eindringen in ein datenverarbeitungssystem.” https://www.fedlex.admin.ch/eli/cc/54/757_781_799/de#art_143_bis, 2024.

[23] Swiss Federal Council, “Art. 144bis stgb datenbeschädigung.” https://www.fedlex.admin.ch/eli/cc/54/757_781_799/de#art_144_bis, 2024.

[24] Swiss Federal Council, “Federal act on data protection (fadp).” https://www.fedlex.admin.ch/eli/cc/2022/491/en, 2023.

Tools and Aids

1. Claude, version claude-sonnet-4-6, Anthropic: https://claude.ai
   Consulted by group members for clarification of open questions, debugging and deeper understanding of selected topics.

2. ChatGPT, version GPT-5.5, OpenAI https://chatgpt.com/
   Consulted by group members to better understand deeper topics and for find and fix bugs.

Appendix A: Ethical & Legal Considerations

Developing ransomware simulation software entails serious legal and ethical responsibilities. This project is strictly for educational and research purposes, and its real-world implications must be clearly understood.

Legal Framework

Ransomware and file encryption attacks are regulated by the Swiss Criminal Code and Data Protection laws:

- **Art. 143 StGB** Unauthorized access to data: Accessing or obtaining data from a computer system without authorization is a criminal offense, regardless of intent. [21]
- **Art. 143bis StGB**Unauthorized access to data processing systems: Infiltrating a computer system without authorization, as a ransomware client would do, is punishable by law. [22]
- **Art. 144bis StGB** Damage to data: Encrypting, modifying or destroying data belonging to another person without authorization is explicitly criminalized. This is the most directly applicable article to ransomware. [23]
- **Federal Act on Data Protection (FADP)**: Collection and processing of personal data requires informed consent and must follow principles of proportionality and purpose limitation. [24]

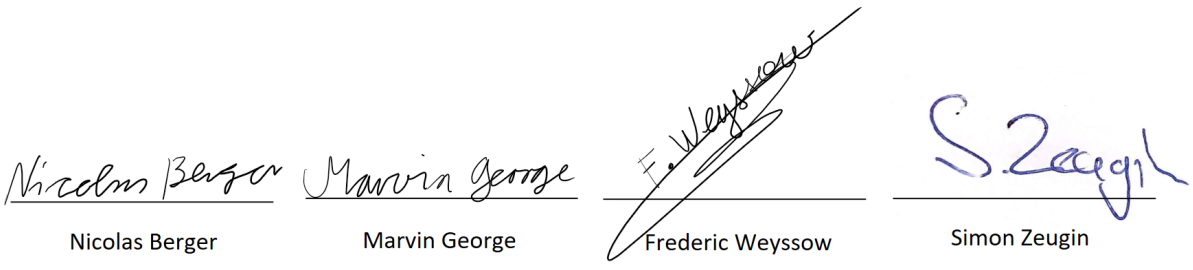
Using such software without explicit consent can lead to fines or imprisonment of up to five years under Art. 144bis StGB.

Appendix B: Division of Work

Frederic: Server/Client architecture, multi threading, key management, encryption, GUI.
Simon: Multi threading, File queue, encryption, Watcher, GUI.
Nicolas: Encryption, Security Component, User/kernel space communication, GUI.
Marvin: Encryption, Security Component, User/kernel space communication, GUI.
Everyone: Formulate new implementation ideas, support when a teammate had problems, debugging, testing.

Appendix C: Declaration of Independent Authorship

"I attest with my signature that I have completed this paper independently and without any assistance from third parties and that the information concerning the sources used in this paper is true and complete in every respect. All sources that have been quoted or paraphrased have been referenced accordingly. Additionally, I affirm that any text passages written with the help of AI-supported technology are marked as such, including a reference to the AI-supported program used. This paper may be checked for plagiarism and use of AI-supported technology using appropriate software. I understand that unethical conduct may lead to a grade of 1 or "fail" or to expulsion from the course of studies. I have taken note of the fact that in the event of a justified suspicion of the unauthorized or undisclosed use of AI in written performance assessments, I am upon request obligated to cooperate in confirming or ruling out the suspicion, for example by attending an interview"



Nicolas Berger | Marvin George | Frederic Weyssow | Simon Zeugin